

Git Commands — Complete Practical Guide

المرجع العملي الشامل لـ Git: من الصفر لحد الاحتراف والـ Internals من غير رغي ولا حشو

Based on Git Official Documentation & Real-World Best Practices



Table of Contents

1. Git Fundamentals & Core Concepts	01	2. Git Setup & Configuration	02
3. Creating & Getting Repositories	03	4. Basic Workflow & Snapshotting	04
5. Git Status & File Lifecycle	05	6. Git Commit & Message Conventions	06
7. Git Log & History Exploration	07	8. Git Show, Describe & Shortlog	08
9. Git Branches (checkout vs switch)	09	10. Git Merge & Conflict Resolution	10
11. Git Rebase (Standard & Interactive)	11	12. Squashing Commits Guide	12
13. Git Stashing Deep Dive	13	14. Remote Repositories Management	14
15. Git Push & Safety Protocols	15	16. Git Fetch Mechanics	16
17. Git Pull (fetch+merge vs fetch+rebase)	17	18. Git Cherry-Pick in Action	18
19. Git Reset vs Git Revert	19	20. Git Diff & Patch Inspection	20
21. Git Grep Codebase Search	21	22. Git Blame & Code Archeology	22
23. Git Bisect Binary Search Debugging	23	24. Git Clean & Safe Untracked Removal	24
25. Git Worktree Multiple Working Trees	25	26. Git Tags & Release Management	26
27. Git Notes Metadata Tracking	27	28. Git Archive Project Packaging	28
29. Git Apply Patch Management	29	30. Git Format-Patch Workflows	30
31. Git Am Mailbox Patching	31	32. Git Range-Diff Commit Series Comparison	32
33. Git Merge-Base Common Ancestors	33	34. Git Rev-Parse Plumbing Evaluation	34
35. Git Reflog Safety Net Mastery	35	36. Git FSCK Repository Health Check	36
37. Git GC & Packfile Optimization	37	38. Git Prune Low-Level Cleanup	38
39. Git Verify-Commit & Cryptographic Signing	39	40. Git Filter-Repo History Rewriting	40
41. Git Submodules Architecture	41	42. Git Subtree Modular Integration	42
43. Git LFS Large File Storage	43	44. Git Bundle Offline Distribution	44
45. Git Archive vs Source Export	45	46. Git Check-Ignore Rule Debugging	46
47. Git LS-Files Index Inspection	47	48. Git LS-Tree Internal Inspection	48
49. Git Show-Ref References Inspector	49	50. Git For-Each-Ref Scripting Tool	50
51. Git Cat-File Internal Objects	51	52. Git Hash-Object SHA Hashing	52
53. Git Update-Ref Low-Level Pointers	53	54. Git Read-Tree Low-Level Indexing	54
55. Git Write-Tree Snapshotting	55	56. Git Commit-Tree Object Creation	56

57. Git Count-Objects Storage Metrics	57	58. Git Verify-Pack Compression Check	58
59. Git Check-Ref-Format Naming Validation	59	60. Git Hooks & CI Automation	60
61. Git Ignore & Secrets Security	61	62. Git Attributes & EOL Handling	62
63. Git Aliases Productivity Powerups	63	64. Git Workflows Comparison	64
65. Real Backend .NET Web API Scenario	65	66. Git Command Classification Matrix	66
67. Comprehensive Command Cheat Sheet	67	68. Top 21 Commands to Master	68
69. Common Git Mistakes & Solutions	69	70. Final Git Mental Model Architecture	70

يعني إيه Git أصلاً؟ Git ده عبارة عن (Distributed Version Control System). عمله لينوس تورفالدس (نفس الراجل اللي عمل كيرنل لينكس) في 2005 عشان يحل مشكلة إدارة كود لينكس الضخم بعد ما اتخانق مع الشركة اللي كانت بتقدم لهم برنامج إدارة النسخ زمان. الميزة الأكبر في Git إنه نظام "موزع"، يعني نسختك اللي على لابتوبك فيها الداتا بيز والتاريخ كله كامل من غير ما تحتاج نت خالص.

إيه الفرق بين Git و GitHub / GitLab ؟

- **Git:** ده التول أو الأداة اللي شغالة عندك على الجهاز في ال Terminal، بتسجل التغييرات وتعمل برانشات وكوميتات أوفلاين تماماً.
- **GitHub / GitLab / Azure DevOps:** دي منصات ومواقع على الكلاود بتستضيف الريبو بتاعك، وبتديك مميزات تيم زي إنك تعمل Pull Request و Code Review وتعمل CI/CD وتتبع ال Issues.

المناطق الأربعة في معمارية Git (لازم تفهمهم صح):

2. Staging Area (منطقة التحضير - ال Index)

منطقة وسيطة ذكية، ملف باينري جوه `.git/index`. دي اللي بتنقي فيها التعديلات اللي خلاص جهزتها وعازيز تدخلها في الكوميت الجاي.

1. Working Directory (ال فولدر اللي شغال فيه)

ده الفولدر العادي بتاع المشروع على جهازك اللي بتفتحه في ال VS Code أو Visual Studio وتكتب فيه كودك وتعدل الملفات براحتك.

4. Remote Repository (الريبو اللي على السيرفر)

النسخة السحابية المرفوعة على GitHub أو السيرفر، واللي التيم كله بيعتلقها (Push) وينزل منها (Pull) عشان يجمعوا شغلهم.

3. Local Repository (الداتا بيز اللي على جهازك)

فولدر `.git` المخفي في مشروعك. ده فيه كل الكوميتات وكل الهيستوري والبرانشات متخزنة ومضغوطة بكفاءة عالية جداً.

مصطلحات هتسمعها كل يوم في الشغل:

- **Commit:** لقطة (Snapshot) كاملة لحالة المشروع في لحظة معينة، ويتأخذ كود مميز (SHA Hash).
- **Branch:** مؤشر بيتحرك معاك (Pointer حجمه 41 بايت بس) بيشاور على أحدث كوميت في خط تطوير معين، ومش بياخد مساحة زيادة من الهارد.
- **HEAD:** المؤشر اللي بيعرف Git أنت واقف فين دلوقتي حالاً على أنهي برانش وأنهي كوميت في الكود.
- **Tag:** علامة ثابتة مبتمشي على كوميت معين، بنستعملها عشان نثبت أرقام الإصدارات الرسمية للبرنامج زي `v1.0.0`.
- **Remote:** اسم اختصار لعنوان السيرفر البعيد (زي `origin` أو `upstream`).
- **Origin:** الاسم الافتراضي اللي Git بيسميه للينك السيرفر اللي عملت منه Clone أول مرة.

مخطط حركة الكود بين مساحات Git الأربعة:



الشكل 1.1: إزاي التعديلات بتتحرك بين مساحات Git

Setup

2. Git Setup & Configuration — تطبيق وإعداد البيئة

أمر `git config` ده اللي بتتطبق بيه إعدادات Git على لابتوبك.

مستويات الإعدادات الثلاثة (Scopes):

المستوى (Scope)	المسار على جهازك	تأثيره	مين يغلب مين؟
--system	C:\Program Files\Git\etc\gitconfig	بيتطبق على كل اليوزرز وكل المشاريع في الجهاز.	الأضعف (Lowest)
--global	~/.gitconfig (في مجلد اليوزر بتاعك)	بيتطبق على كل مشاريعك أنت شخصياً على اللابتوب.	الوضع الطبيعي اللي بنستخدمه
--local	.git/config (جوه فولدر المشروع ده بس)	بيتطبق على المشروع ده تحديداً وبلغي أي إعداد عام.	الأعلى أولوية (Highest)

تطبيق إعدادات أول مرة بعد ما تسطب Git:

أوامر التطبيق السريع

```
# 1. تأكد من النسخة اللي عندك
git --version

# 2. اكتب اسمك وإيميلك (دول اللي هيتلحقوا في كل كوميت تعمله)
git config --global user.name "Amar Developer"
git config --global user.email "amar@example.com"

# 3. Rebase كمحرر افتراضي عشان يفتح وقت الرسايل والـ VS Code اختار الـ
git config --global core.editor "code --wait"

# 4. main خلي البرانش الافتراضي في أي مشروع جديد اسمه
git config --global init.defaultBranch main

# 5. عشان متعملش قفلة مع الـ لينكس والدوكر (CRLF) ظبط نهايات الأسطر للويندوز
git config --global core.autocrlf true

# 6. اعرض كل الإعدادات ومكان ملف كل إعداد منهم
git config --list --show-origin

# 7. افتح صفحة المساعدة لأي كوماندا
git help commit
git commit -h

# 8. فيه مشكلة في النظام Git طلع تقرير بالأخطاء لو
git bugreport
```

⚠ حركة مهمة جداً

اتأكد إن الإيميل اللي كتبت في `user.email` هو نفس الإيميل اللي عامل بيه حسابك على GitHub، عشان الكوميتات تسمع في بروفيلك والمربعات الخضرا تظهر!

Repo Creation

3. Creating & Getting Repositories — إنشاء وتنزيل المشاريع

1. أمر `git init` (تبدأ مشروع جديد من الصفر):

بيحول الفولدر اللي أنت واقف جواه لمشروع Git شغال، عن طريق إنه يكرت فولدر مخفي اسمه `.git`.

تهيئة مستودع جديد

```
# Git تحويل الفولدر الحالي لمستودع
git init

# مرة واحدة Git تفتح فولدر جديد وتبدأ فيه
git init my-dotnet-app
```

إيه اللي مستخبي جوه فولدر `.git` ده؟

- `HEAD` : ملف نصي صغير بيقول لـ Git أنت واقف على أي برانش دلوقتي حالا.
- `config` : ملف الإعدادات الخاصة بالمشروع ده بالذات.

- `hooks/` : فولدر السكريبتات اللي بتشتغل أوتوماتيك مع الكوميت وال Push.
- `objects/` : قلب الداتا بايز اللي بيتخزن فيه كل الملفات والهيستوري والكوميتات مضغوطة.
- `refs/` : الفولدر اللي بيثيل المؤشرات (الفروع المحلية heads، والإصدارات tags، والفروع البعيدة remotes).

2. أمر `git clone` (تنزيل مشروع موجود على السيرفر):

بينزل نسخة كاملة من المشروع بكل فروعه وتاريخه من أول كوميت اتعمل فيه لحد آخر ثانية على لابتوبك.

Clone طرق عمل

```
# تنزيل المشروع في فولدر بنفس اسمه
git clone https://github.com/company/ECommerceApp.git

# تنزيل المشروع في فولدر باسم تختاره أنت
git clone https://github.com/company/ECommerceApp.git MyLocalFolder

# (Shallow Clone) وسيرفرات البيلد CI/CD تنزيل آخر لقطة بس عشان تسرع الـ
git clone --depth 1 --branch main https://github.com/company/ECommerceApp.git
```

Porcelain

4. Basic Workflow / Snapshotting — دورة الشغل اليومية وحفظ اللقطات

دي الأوامر اللي إيدك هتاخذ عليها كل يوم في الشغل:

- `git status` : بتعرفك مين اتعدل ومين جاهز ومين لسه Git ميعرفوش.
- `git add <file>` : بتاخذ الملفات من الكود وتحطها في ال Staging Area تجهيزاً للكوميت.
- `git add .` أو `git add -A` : بتجهز كل التعديلات والملفات الجديدة والمحذوفة في المشروع كله دفعة واحدة.
- `git add -p` : التجهيز الذكي؛ بتراجع التعديلات حته حته وتختار إيه اللي يدخل وإيه اللي يستنى.
- `git commit -m "msg"` : تسجيل اللقطة بشكل دائم جوه داتا بايز Git المحلية.
- `git diff` : بيوريك التغييرات اللي في الكود ولسه مجهّزتهاش في ال Staging.
- `git diff --staged` : بيوريك التعديلات اللي جهزتها في ال Staging ومستعدة للكوميت.
- `git restore <file>` : بيلغي تعديلك على الملف ويرجعه زي ما كان في آخر كوميت.
- `git restore --staged <file>` : بينزل الملف من ال Staging بس يسبب تعديله في كودك عادي.
- `git rm <file>` : بيمسح الملف من كودك ومن تتبع Git مع بعض.
- `git rm --cached <file>` : بيثيل الملف من تتبع Git بس يسببه على جهازك (حركة منقذة لما تنسى تحط ملف في ال `.gitignore`).
- `git mv <old> <new>` : بينقل أو يغير اسم ملف ويجهزه تلقائياً في ال Staging.

File Lifecycle

5. Git Status & Changes — حالات الملفات في Git

أي ملف في مشروعك بيمر بـ 4 حالات أساسية:

1. Untracked (ملف جديد لسه متعرفش)

ملف جديد لسه عامله حالا، Git شايفه موجود على الهارد بس ميعرفش أي تفاصيل عنه ولسه مبيسجلش تاريخه.

2. Modified (متعدل بس مش جاهز)

ملف قديم Git عارفه، فتحته وعدلت فيه أسطر ولسه معملتش `git add`.

3. Staged (جاهز للكوميت)

ملف عملته `git add` ومستعد إنه يتحفظ في الكوميت الجاي فوراً.

4. Committed (محفوظ في الأمان)

الملف اتسجل خلاص في داتابيز Git، والنسخة اللي في كودك مطابقة لآخر لقطة.



❗ سؤال يجير ناس كثير: إيه اللي يحصل لو عدلت ملف بعد ما عملت `git add`؟

هتلاقي `git status` مطلعة الملف في مكانين مع بعض! جزء منه Staged وجزء منه Modified. لو عملت Commit دلوقتي، Git هياخد النسخة اللي كانت جاهزة بس! عشان كده لازم تعمل `git add` ثاني لو عايز آخر تعديل يدخل معاك.

Porcelain

6. Git Commit & Messages — تسجيل الكوميتات باحتراف

أمر `git commit` بياخد اللقطة المتجهزة ويسجلها في الداتابيز، ويربطها بالكوميت اللي قبلها، ويسجل اسمك وتاريخك والرسالة بتاعتك.

Commit أوامر الـ

```
# تسجيل الكوميت العادي مع رسالة واضحة
git commit -m "feat(auth): implement JWT token generation"

# (Untracked مش هيشمل الملفات الجديدة) تجهيز وتسجيل الملفات المتعدلة مرة واحدة
git commit -a -m "fix(order): handle null exception in calculate total"

# تعديل آخر كوميت محلي لو نسيت ملف أو عايز تصلح رسالته
git commit --amend -m "feat(auth): implement JWT token with refresh token"

# تزويد ملف نسيته على آخر كوميت من غير ما تغير الرسالة
git add ForgottenFile.cs
git commit --amend --no-edit
```

إزاي تكتب رسالة كوميت محترمة (Conventional Commits)؟

رسائل مهندسين شاطرين ✓

- feat(payment): add Stripe webhook handler
- fix(api): resolve 500 error on empty cart
- refactor(db): optimize user query indexing
- docs(readme): add docker-compose guide

رسائل عشوائية بلاش تكتبها ✗

- fix bug (باج إيه وفين بالضبط؟)
- update (مش مفهوم أي حاجة)
- changes by Amar (اسمك مسجل أصلاً)
- wip (ممنوع ترفع كده على فروع التيم)

⚠ تحذير عالي الخطورة بخصوص git commit --amend

إياك تعمل `--amend` على كوميت أنت عملته `git push` خلاص على فرع مشترك مع التيم! لأن الـ `amend` بيغير الهاش تماماً ويعيد كتابة التاريخ وده هيبوظ الدنيا عند زمايلك. استخدمه على لابتوبك بس قبل ما تعمل `Push`.

Inspection

7. Git Log & History — قراءة واستكشاف تاريخ المشروع

أمر `git log` هو عينك اللي بتشوف بيها تاريخ الشغل والكوميتات اللي اتعملت في المشروع.

git log أهم أوامر

عرض السجل العادي

git log

عرض كل كوميت في سطر واحد مختصر ونظيف

git log --oneline

رسم شجرة البرانشات بالألوان

git log --graph --oneline --decorate --all

عرض إحصائية الملفات اللي اتعدلت وعدد السطور

git log --stat

عرض التعديلات سطر بسطر لآخر كوميتين

git log -p -2

تصفية الكوميتات باسم ديفيلوير معين

git log --author="Amar"

تصفية التاريخ بوقت معين

git log --since="2.weeks.ago" --until="yesterday"

التدوير في رسائل الكوميتات على كلمة معينة

git log --grep="JWT" --oneline

شكل شجرة الفروع في ال Terminal:

```
* d8a4f12 (HEAD -> main, origin/main) feat(order): add invoice export
* 7c2b3e4 Merge branch 'feature/auth' into main
|\
| * 9f1a2b3 (feature/auth) feat(auth): add password hashing with BCrypt
| * 4e5d6c7 feat(auth): create Login and Register DTOs
|/
* 1a2b3c4 chore: initial ASP.NET Core project setup
```

الشكل 7.1: سجل تاريخ تفاعلي بيوضح تفرع ميزة ال Auth ودمجها في main

Inspection

8. Git Show, Describe & Shortlog — فحص الكائنات والتقارير

1. أمر `git show`:

يفتح لك تفاصيل كاملة لكوميته أو تاج معين ويوريك ال Diff بتاعه سطر بسطر.

استخدامات `git show`

عرض تفاصيل آخر كوميته

```
git show HEAD
```

عرض محتوى ملف معين في كوميته قديم من غير ما تبدل مكانك

```
git show HEAD~2:src/Controllers/AuthController.cs
```

2. أمر `git describe`:

بيطلعك اسم بشري مقروء للكوميته بناءً على أقرب Release Tag، وده بنستعمله في سيرفرات ال CI/CD عشان نطلع رقم الإصدار تلقائياً.

استخراج رقم الإصدار

```
git describe
```

```
# c2b3e4 والهاش الحالي 7 v1.2.0 معناها: 4 كوميتات بعد التاج (v1.2.0-4-g7c2b3e4): الناتج
```

3. أمر `git shortlog`:

يلخص كل واحد في التيم عمل كام كوميته، حلو جداً في تقارير السبرنت:

تقرير مساهمات التيم

```
git shortlog -sn --no-merges
```

الناتج:

```
# 42 Amar Developer
```

```
# 18 Sarah Engineer
```

```
# 7 Omar Backend
```

Branching

9. Branches — إدارة الفروع والتنقل (Checkout vs Switch)

البرانش في Git مش نسخة ثانية من الفولدر، ده مجرد مؤشر خفيف جداً (Pointer حجمه 41 بايت) بيشار على الكوميته. التفرع في Git سريع جداً ومبيأخدش مساحة خالص.

أوامر إدارة الفروع

```
# عرض الفروع المحلية (والفرع اللي أنت واقف عليه لونه أخضر وعليه نجمة)
git branch

# (Remote المحلية وفروع السيرفر) عرض كل الفروع
git branch -a

# تعمل برانش جديد وتفضل واقف مكانك
git branch feature/payment-gateway

# (Safe Delete) مسح برانش مدموج وخلصت منه
git branch -d feature/payment-gateway

# (Force Delete) إجبار مسح برانش حتى لو فيه كود مدمجوش
git branch -D feature/experimental-test

# تغيير اسم البرانش اللي أنت واقف عليه
git branch -m feature/new-auth-v2
```

المقارنة التاريخية: `git checkout` مقابل `git switch` :

زمان كان أمر `git checkout` يعمل كذا حاجة ملهاش علاقة ببعض ويلخبط الناس (يتنقل بين الفروع، ويعمل فروع، ويرجع ملفات قديمة). في إصدار Git 2.23 فصلوا الموضوع ده بوضوح تام:

- `git switch` : مخصص بس للتنقل بين الـ Branches وإنشائها.
- `git restore` : مخصص بس لإلغاء وتعديل واسترجاع الملفات.

التبديل بين الفروع

```
# الطريقة الحديثة والواضحة (Recommended):
git switch feature/login          # تروح لبرانش موجود
git switch -c feature/orders-service # تعمل برانش جديد وتروح عليه فوراً
git switch -                      # ترجع بسرعة لآخر برانش كنت واقف عليه

# الطريقة القديمة (Legacy):
git checkout feature/login
git checkout -b feature/orders-service
```

Merging

10. Git Merge — دمج الفروع وحل النزاعات (Conflicts)

عملية الـ Merge بتجمع شغل فرعين مختلفين وتدمجهم في فرع واحد.

أنواع الـ Merge في Git:

2. Three-Way Merge

بيحصل لما يكون الفرعين دخل عليهم كوميتات جديدة في نفس الوقت. Git بيشوف الأصل المشترك ويعمل "Merge Commit" خاص بيكون فيه أبوين (Two Parents).

1. Fast-Forward Merge

بيحصل لما الفرع الأساسي (main) ميكونش اتضاف عليه أي كود جديد من وقت ما فرعت منه. Git بيمشي المؤشر لقدام وخلص من غير ما يعمل كوميت دمج جديد.

```

=== 1. قبل الدمج ===
(main) -> A --- B
              \
              C --- D <- (feature/login)

=== 2. Fast-Forward Merge ===
(main, feature/login) -> A --- B --- C --- D

=== 3. Three-Way Merge (E زاد عليه كود جديد main لما) ===
(main) -> A --- B --- E
              \      \
              C --- D --- M (Merge Commit: أبوين D و E)

```

الشكل 10.1: المقارنة بين Fast-Forward و Three-Way Merge

أوامر الدمج وخياراتها

```

# 1. تروح للفرع اللي عايز تدمج فيه الأول
git switch main

# 2. تدمج الفرع الثاني
git merge feature/login

# 3. (عشان تحافظ على شكل الميزة في التاريخ) Fast-forward حتى لو ينفع Merge Commit إجبار عمل
git merge --no-ff feature/login

# 4. وعمايز ترجع للوضع النظيف الأولاني Conflicts إلغاء الدمج بالكامل لو ظهرت
git merge --abort

```

History Rewriting

11. Git Rebase — إعادة التأسيس والدمج التفاعلي

الـ Rebase فكرته ببساطة: بدل ما تعمل Git Merge Commit، بياخد الكوميترات اللي أنت عملتها في فرعك، ويشيلها يحطها فوق أحدث حاجة نزلت على `main`، فيطلع التاريخ خط مستقيم ونظيف جداً (Linear History) بدون تعقيدات الدمج.

```

=== Rebase قبل الـ ===
main:    A --- B --- C
          \
feature:  D --- E (B مبني على البداية القديمة)

=== feature على فرع git rebase main بعد تنفيذ ===
main:    A --- B --- C
          \
feature:  D' --- E' (بهاشات جديدة C اتطبقوا فوق)

```

الشكل 11.1: إزاي الـ Rebase بينقل الكوميترات فوق أحدث نقطة

الـ Interactive Rebase (git rebase -i) لتنظيف التاريخ:

أقوى أداة لتنظيم وتعديل ومسح ودمج الكوميترات قبل ما ترفع الـ Pull Request:

بدء Interactive Rebase

```

# فتح التعديل التفاعلي لآخر 4 كوميترات
git rebase -i HEAD~4

```

أوامر ال Interactive Rebase المتاحة:

الأمر	ببعمل إيه؟	تأثيره
<code>pick (p)</code>	سيب الكوميت ده زي ما هو من غير تغيير.	ببفضل بنفس محتواه
<code>reword (r)</code>	سيب محتوى الكود زي ما هو بس افتحلي أعدل الرسالة.	ببغير الهاش
<code>edit (e)</code>	وقفني عند الكوميت ده أعدل في الكود أو أقسمه كذا كوميت.	تعديل عميق
<code>squash (s)</code>	ادمج الكوميت ده في اللي قبله واجمع رسايلهم مع بعض.	بيدمج ويولد هاش جديد
<code>fixup (f)</code>	ادمج الكوميت ده في اللي قبله وامسح رسالته خالص (تنظيف سريع).	تنظيف صامت وممتاز
<code>drop (d)</code>	امسح الكوميت ده خالص من التاريخ ولا كأنه اتعمل.	إزالة نهائية للكود

جدول المقارنة الحاسم: Merge ضد Rebase:

وجه المقارنة	Git Merge	Git Rebase
شكل التاريخ	شجري غير خطي، بيحتفظ بالتاريخ الحقيقي بدقة.	خطي مستقيم ونظيف جداً وسهل تقرأه.
إنشاء Merge Commit	نعم (في ال Three-way).	لا، مفيش أي Merge Commits زيادة.
إعادة كتابة التاريخ	آمن تماماً، مبيغيرش هاشات قديمة.	بيعيد كتابة وحساب الهاشات كلها.
الفروع المشتركة (Shared)	آمن ومطلوب للفروع العامة زي <code>main</code> .	ممنوع وخطر جداً على الفروع المشتركة.
حل التعارضات	بيتحل مرة واحدة في ال Merge Commit.	ممکن تحل تعارض في كل كوميت بيتنقل.
نسبة الخطورة	منخفضة جداً (Safe).	عالية لو المطور مش فاهم ببعمل إيه.

History Cleanup

12. Squashing Commits — ضغط وتنظيف الالتزامات

وأنت شغال على Feature معينة، طبيعي بتعمل كوميتات كتير صغيرة ومؤقتة زي: `fix typo` , `test bug` , `fix validation` . قبل ما تبعت الشغل للتيم في PR، الصح إنك تعمل **Squash** وتدمجهم كلهم في كوميت واحد نظيف ومعبر.

سيناريو عملي لدمج 4 كوميتات في كوميت واحد

```
# فتح التعديل التفاعلي لآخر 4 كوميتات 1.
git rebase -i HEAD~4

# هيفتحك المحرر بالشكل ده:
# pick a1b2c3d feat(auth): add login endpoint
# squash e4f5g6h fix validation logic
# squash i7j8k9l fix typo in model
# fixup m0n1o2p fix build warning

# هيطلب منك رسالة مجمعة نظيفة واحدة Git، بعد ما تحفظ وتقفل
# feat(auth): implement complete user authentication and validation
```

أمر `git stash` عامل زي درج مكتبك؛ بياخد كل التعديلات اللي لسه شغال عليها ومش جاهزة تعملها كوميت، ويشيلها على جنب في مكدس (Stack) ويرجعك المشروع نظيف تماماً زي آخر كوميت، عشان تعرف تبدل الفروع وترجع لشغلك بعدين.

Stash أوامر الـ

```
# 1. ركن التعديلات مع كتابة رسالة توضح كنت بتعمل إيه
git stash push -m "WIP: cart checkout logic partially done"

# 2. Untracked ركن التعديلات مع الملفات الجديدة اللي لسه
git stash -u

# 3. Stash عرض قائمة الحاجات المكونة في الـ
git stash list

# 4. (Pop) استرجاع آخر تعديلات مكونة ومسحها من الدرج
git stash pop

# 5. Stash (Apply) استرجاع التعديلات بس تسيب نسخة منها في الـ
git stash apply stash@{0}

# 6. معين stash مسح
git stash drop stash@{0}

# 7. نهائياً Stashes مسح كل الـ
git stash clear
```

سيناريو واقعي من بيئة الشغل: باج طارئ في الإنتاج (Production Hotfix):

Stash التعامل مع الطوارئ بـ

```
:وجالك تليفون إن في عطل في البرودكشن feature/orders أنت شغال على
# 1. اركن الشغل الحالي على جنب
git stash push -u -m "WIP: orders work"

# 2. وصلح الباج main روح للـ
git switch main
git switch -c hotfix/payment-fix
... صلح الكود واعمل تيسر ...
git commit -am "fix(payment): resolve null currency conversion"
git push -u origin hotfix/payment-fix

# 3. !ارجع لميزتك واسترجع شغلك ثاني زي ما سبته بالضبط
git switch feature/orders
git stash pop
```

ال Remotes هي السيرفرات السحابية اللي بنرفع عليها المشروع عشان نتشارك الشغل مع باقي التيم.

Remotes إدارة الـ

```
# عرض أسماء وعناوين السيرفرات المربوطة
git remote -v

# Open Source في مشاريع الـ upstream زي ربط الـ) ربط سيرفر بعيد جديد
git remote add upstream https://github.com/original-org/CorePlatform.git

# عرض تفاصيل حالة مزامنة الفروع مع السيرفر
git remote show origin

# مسح ربط سيرفر بعيد
git remote remove upstream
```

Remote Sync

15. Git Push & Safety — رفع الشغل وقواعد الأمان الحساسة

أمر `git push` بيرفع الكوميتات والمؤشرات من جهازك للسيرفر السحابي.

الأساسية Push أوامر الـ

```
# التلقائي Tracking رفع الفرع لأول مرة وتطبيق الـ
git push -u origin feature/login

# (عارف هيرفع فين أوتوماتيك Git) عمليات الرفع اللي بعد كده
git push

# والإصدارات للسيرفر Tags رفع الـ
git push --tags
```

المقارنة الخطيرة: `--force` ضد `--force-with-lease` :

2. `git push --force-with-lease` (المعيار الآمن)

بيتشيك الأول؛ لو لقي زميلك رفع حاجة جديدة أنت لسه منزلتهاش على جهازك، الرفع هيفشل فوراً ويحميك من إنك تطير شغل غيرك بالخطأ.

1. `git push --force (-f)` (كارثة)

بيسمح أي كود موجود على السيرفر ويحط النسخة اللي على جهازك غصب عن أي حد! لو زميلك رفع كود جديد قبلك، الـ force هتمسح شغله وتضيعه نهائياً!

Remote Sync

16. Git Fetch — جلب التعديلات بأمان من غير دمج

أمر `git fetch` بينزل كل الكوميتات والفروع الجديدة من السيرفر لجهازك، بس ميبلمسش ولا يغير أي سطر في ملفاتك اللي شغالة في الكود. يعني تقدر تبص على شغل الناس بأمان 100% من غير ما يحصل أي لخبطة في كودك.

git fetch أوامر

```
# جلب التعديلات من السيرفر الأساسي
git fetch origin

# جلب التعديلات من كل السيرفرات المربوطة
git fetch --all

# (Prune) جلب التعديلات ومسح أسماء الفروع التي اتمسحت خلاص على السيرفر
git fetch --all --prune
```



الشكل 16.1: إزاي git fetch بي فصل بين تنزيل المؤشرات وبين الدمج الفعلي

Remote Sync

17. Git Pull — جلب ودمج التعديلات التلقائي

أمر `git pull` هو في الحقيقة أمر مركب بيعمل خطوتين ورا بعض تلقائياً:

`git pull = git fetch + (git merge OR git rebase)`

git pull طرق عمل

```
# 1. (ملهاش لازمة Merge Commit مع إنشاء Fetch 3-Way Merge) الطريقة القديمة
git pull --no-rebase

# 2. (عشان تحافظ على تاريخ خطي نظيف Rebase ثم Fetch) الطريقة الاحترافية الحديثة
git pull --rebase

# 3. أوتوماتيك rebase دائماً شغال بنظام pull يخلي الـ Git تطبيط
git config --global pull.rebase true
```

Commit Management

18. Git Cherry-Pick — خطف ونقل Commits معينة

أمر `git cherry-pick` بيسمهلك تنقي كومييت واحد بعينه من فرع ثاني وتأخده تطبقه عندك في فرعك الحالي، من غير ما تضطر تدمج الفرع الثاني بكل اللي جواه.

```

main:          A --- B --- C -----> (HEAD)
                  \
feature/v2:     D --- E (Bugfix) --- F
                  ^
                git cherry-pick E
                  v
main (بعد الخطف): A --- B --- C --- E' (اتطبق بهاش جديد تماماً)

```

الشكل 18.1: خطف ال Commit E من فرع التطوير وتطبيقه على فرع الإنتاج

سيناريو واقعي في الشغل: تصليح باج سريع في البرودكشن (Production Hotfix):

زميلك صلح باج أمني خطير في فرع `feature/auth-v2` في كوميت رقمه `(e4a19bc)`. الفرع ده مليون كود لسه متجربش ومش جاهز ينزل، بس الإنتاج (main) محتاج التصليحة دي حالا:

Cherry-Pick تطبيق

```

# 1. روح لفرع الإنتاج الرئيسي.
git switch main

# 2. اخطف الكوميت المطلوب بس.
git cherry-pick e4a19bc

# cherry-pick: وأنت بتعمل Conflict لو ظهر
git status # اعرف الملفات المتعارضة
# ثم: VS Code صلح الملفات في الـ
git add src/Services/AuthService.cs
git cherry-pick --continue # كمل العملية

# لو عايز تلغي وترجع زي ما كنت:
git cherry-pick --abort

```

History & Undo

19. Git Reset vs Git Revert — الرجوع في كلامك والتراجع الآمن

ده من أهم وأخطر المفاهيم؛ الخلط بينهم ممكن يطير شغل التيم. الفكرة كلها: هل عايز تمسح التاريخ وتعيد كتابته، ولا عايز تعمل Commit جديد يعكس التراجع؟

1. أمر `git reset` (محلي بس + بيمسح التاريخ القديم):

بيرجع مؤشر `HEAD` لورا لكوميت سابق، وفيه 3 أوضاع مشهورة:

- `--soft`: يرجعك لورا، بس يسيب كل التعديلات الملفات متجهزة في ال **Staging Area**. ممتاز لو عايز تدمج كذا كوميت عملتهم ورا بعض.
- `--mixed` (الوضع الافتراضي): يرجعك لورا، وينزل التعديلات من ال **Staging**، بس يسيب الملفات موجودة في الكود كملفات معدلة.
- `--hard`: بيمسح ويلغي كل التعديلات نهائياً من الوجود ويرجع المشروع زي ما كان في الكوميت ده بالملي (خطر جداً لو مش عامل حسابك!).

الثلثة Reset أنواع الـ

```
# Staging تراجع مع بقاء التعديلات جاهزة في الـ
git reset --soft HEAD~1

# تراجع مع بقاء التعديلات كملفات عادية في المشروع
git reset --mixed HEAD~1

# تراجع ومسح كل التغييرات نهائياً (احذر منه!)
git reset --hard HEAD~1
```

2. أمر git revert (آمن جداً ومخصص للفروع المشتركة):

مبمسحش أي حاجة من التاريخ؛ بل بينشئ **Commit** جديد تماماً بيعمل عكس التعديل اللي انت حددته عشان يلغي تأثيره، فالتاريخ يفضل سليم ومحدث من زمالك يتلخبط.

git revert استخدام

```
# إلغاء تأثير كوميت معين بعمل كوميت جديد مضاد ليه
git revert d8a4f12
```

جدول المقارنة الحاسم: revert vs reset vs restore

الأمر	بيشتغل على إيه؟	تأثيره على تاريخ الكوميتات	تستخدمه إمتى؟	هل آمن على الفروع المشتركة؟
git restore	Working Tree & Staging Area	مبيلمسش أي Commits.	إلغاء تعديلات ملفات لسه متسجلتش.	آمن تماماً (Safe)
git reset	HEAD, Index, Working Tree	بيمسح ويعيد كتابة التاريخ.	تنظيم وترتيب شغلك المحلي على لابتوبك بس.	خطر جداً وممنوع على Shared Branches
git revert	بيعمل New Commit جديد	بيحافظ على التاريخ ويضيف لقطة تراجع.	التراجع عن ميزة اترفعت خلاص على main .	المعيار الذهبي والأمن للتيم (Best Practice)

Inspection

20. Git Diff — فحص الفروقات سطر بسطر

أمر git diff بيكشفلك الفروقات والتغييرات الدقيقة بين الملفات والكوميتات سطر بسطر.

git diff استعلامات

```
# 1. Staging Area مقارنة التعديلات التي في الكود مع الـ
git diff

# 2. Commit مع آخر Staging Area مقارنة التعديلات المجهزة في الـ
git diff --staged

# 3. Commit مقارنة الكود الحالي مع آخر
git diff HEAD

# 4. محددتين بالهاش Commits مقارنة بين 2
git diff 1a2b3c4 7c2b3e4

# 5. مقارنة بين فرعين كاملين
git diff main feature/payment

# 6. مقارنة ملف واحد بس بين فرعين
git diff main..feature/payment -- src/Services/OrderService.cs
```

إزاي تقرأ الـ Diff Header وتفهم الرموز:

```
--- a/src/Program.cs      (النسخة القديمة)
+++ b/src/Program.cs      (النسخة الجديدة بعد التعديل)
@@ -14,6 +14,7 @@         (من السطر 14: 6 أسطر قديمة مقابل 7 أسطر جديدة)
  builder.Services.AddControllers();
- builder.Services.AddAuthentication();      (سطر اتمسح - أحمر)
+ builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme); (سطر اضافة - أخضر)
+ builder.Services.AddAuthorization();
  app.MapControllers();
```

الشكل 20.1: تشرح مخرجات Git Diff

Search

21. Git Grep — تدوير وبحث صاروخي جوه الكود

أمر `git grep` محرك بحث سريع جداً بيدور جوه ملفات المشروع والتاريخ المأرشف في ثواني معدودة.

ASP.NET Core في مشروع `git grep` أمثلة البحث —

```
# البحث عن نص مع رقم السطر واسم الملف
git grep -n "IOOrderRepository"

# بحث غير حساس لحالة الأحرف (كبير/صغير)
git grep -i "dbcontext"

# (Regex) بحث بتعبيرات نمطية منتظمة
git grep -E "throw new [A-Za-z]+Exception"

# البحث عن كل تعليقات المهام المعلقة في الكود كله
git grep -n "TODO:"

# قديم من غير ما تنقل الفرع Release البحث جوه
git grep "StripeClient" v1.0.0
```

أمر `git blame` يعرض لك كل سطر في الملف مين المطور اللي كتبه، والكوميت رقمه كام، واتكتب يوم إيه والساعة كام.

استخدامات `git blame`

```
# فحص ملف كامل سطر بسطر
git blame src/Controllers/PaymentController.cs

# فحص نطاق أسطر معينة بس (من السطر 45 إلى 60)
git blame -L 45,60 src/Controllers/PaymentController.cs

# تجاهل تعديلات المسافات والتنسيقات عشان توصل للكاتب الحقيقي للكود
git blame -w -L 45,60 src/Controllers/PaymentController.cs
```

i ثقافة هندسية راقية (Blameless Culture)

الهدف من `git blame` مش إنك تلوم أو تحاسب زميلك على باج، الهدف إنك تفتح رسالة الكوميت وتفهم السياق والسبب المنطقي اللي خلاه يكتب الكود بالطريقة دي!

أمر `git bisect` بيستخدم خوارزمية البحث الثنائي (Binary Search) عشان يصطاد الكوميت اللي دخل باج في وسط مئات الكوميتات في 4 أو 5 خطوات بس $O(\log n)$!!

```
(100 Commits) التاريخ:
[HEAD] باظ هنا -----> [نقطة المنتصف] ----- [v1.0] شغال تمام
|
:بيسألك هنا Git
هل الكود شغال سليم؟
- (دور في النص اليمين) git bisect good -> لو شغال
- (دور في النص الشمال) git bisect bad -> لو بايظ
الشكل 23.1: آلية البحث الثنائي بـ git bisect
```

سيناريو عملي في مشروع .NET:

إنديوينت كانت شغالة طيارة من أسبوعين، والنهاردة بتضرب `500 Error` وفيه 70 كوميت اتعملوا في النص ومحدث عارف مين السبب:

git bisect خطوات تشخيص الباجات بـ

```
# 1. بدء جلسة التحقيق
git bisect start

# 2. تحديد إن النسخة اللي في إيدك حالياً بايطة
git bisect bad

# 3. كانت شغالة زي الفل v1.2.0 تحديد إن النسخة القديمة
git bisect good v1.2.0

# هينقلك لنقطة المنتصف تلقائياً، قم بتشغيل التيست
dotnet test

# 4. بالنتيجة Git بلغ:
git bisect good # لو التيست نجح
# أو:
git bisect bad # لو التيست فشل

# هو اللي بوظ الدنيا واسم المطور ورسالته Commit 7a8b9c: هيقولك بالضبط Git، بعد كام خطوة
# إنهاء التحقيق والرجوع لمكانك
git bisect reset
```

Cleanup

24. Git Clean — تنظيف الملفات الزيادة بأمان

أمر `git clean` بيمسح الملفات غير المتتبعة (Untracked) ومخلفات البناء المتروكة في ال Working Directory.

⚠ تحذير عالي الخطورة

الملفات اللي بتتمسح بـ `git clean` مبتروحش سلة المهملات ومبتتراجعش بـ Git! لازم تشغل التجربة الجافة (Dry-run) بـ `-n` الأول!

أوامر تنظيف المستودع

```
# 1. شوف إيه اللي هيتمسح من غير ما يمسح حاجة: التجربة الجافة الإلزامية (Dry-run)
git clean -nd

# 2. المسح الفعلي للملفات غير المتتبعة
git clean -f

# 3. مسح الملفات والمجلدات غير المتتبعة
git clean -fd

# 4. (bin و obj زي ملفات) .gitignore. مسح حتى الملفات المتجاهلة بـ
git clean -fdx
```

Worktrees

25. Git Worktree — فتح كذا فرع شغالين في نفس الوقت

ميزة `git worktree` بتسمحك تفتح كذا فولدر عمل على الهارد لنفس المشروع وكل فولدر شغال على Branch مختلف في نفس الوقت، من غير ما تحتاج تعمل Stash أو تبدل الفروع في المحرر.

```
# إنشاء مساحة عمل جديدة على فرع جديد للإصلاح السريع
git worktree add ../hotfix-payment -b hotfix/payment-issue

# ثاني وتشغل بالتوازي VS Code تقدر تفتح الفولدر الجديد في شباك!

# استعراض مساحات العمل المفتوحة.
git worktree list

# مسح مساحة العمل بعد ما خلصت الشغل.
git worktree remove ../hotfix-payment

# تنظيف المراجع القديمة.
git worktree prune
```

Release

26. Git Tags — توثيق الإصدارات وال Releases

ال Tags هي علامات ثابتة على كوميتات معينة لتوثيق الإصدارات الرسمية للبرنامج.

2. Annotated Tag (الموصى به للإنتاج)

كائن كامل جواه اسمك وإيميلك وتاريخ ورسالة توثيق كاملة وتوقيع رقمي.

```
git tag -a v1.0.0 -m "Release 1.0.0"
```

1. Lightweight Tag

مجرد مؤشر بسيط على الكوميت بدون أي بيانات إضافية.

```
git tag v1.0.0-beta
```

Tags إدارة الـ

```
# المتاحة Tags عرض الـ
git tag

# فحص تفاصيل التاج
git show v1.0.0

# رفع تاج معين للسيرفر
git push origin v1.0.0

# الجديدة مرة واحدة Tags رفع كل الـ
git push --tags

# مسح تاج محلي
git tag -d v1.0.0-beta

# مسح تاج من على السيرفر البعيد
git push origin --delete v1.0.0-beta
```

Metadata

27. Git Notes — تزويد ملاحظات من غير تغيير ال Hash

أمر `git notes` يسمحك تضيف بيانات وملاحظات لكوميت موجود مسبقاً (زي نتائج فحص الأمان أو تقارير البيلد) من غير ما يغير ال SHA-1 Hash بتاع الكوميت إطلاقاً.

git notes استخدام

```
# تزويد ملاحظة لآخر كوميت
git notes add -m "Passed Security Audit & Static Code Analysis"

# عرض الملاحظة
git notes show d8a4f12

# مسح الملاحظة
git notes remove d8a4f12
```

Packaging

28. Git Archive — تصدير الكود كملف مضغوط Zip

أمر `git archive` يضغط كود المشروع لنسخة أو فرع معين في ملف Zip أو Tar من غير ما يدخل جواه فولدر `.git`. أو التاريخ الداخلي، ممتاز لتسليم الكود النقي للعميل.

git archive تصدير الكود بـ

```
# Zip كملف مضغوط v1.0.0 تصدير الإصدار
git archive --format=zip --output=release-v1.0.0.zip v1.0.0
```

Patches

29. Git Apply — تطبيق ملفات ال Patches

أمر `git apply` يباخذ ملف رقعة (Patch File) ناتج عن `git diff` ويطبقه على الكود مباشرة من غير ما يعمل أي كوميتات في التاريخ.

git apply تطبيق الرقع بـ

```
# اختبار إمكانية تطبيق الباتش من غير أخطاء
git apply --check fix-bug.patch

# تطبيق الباتش فعلياً
git apply fix-bug.patch
```

Email Workflows

30. Git Format-Patch — تصدير رقع جاهزة للإيميل

أمر `git format-patch` يحول الكوميتات لملفات باتش منسقة وجاهزة للإرسال عبر البريد الإلكتروني (النظام المعتمد في نواة لينكس ومشاريع ال Open Source الضخمة).

توليد الباتشات البريدية

```
# لآخر 3 كوميتات patch توليد ملفات
git format-patch -3 HEAD
```

Email Workflows

31. Git Am — تطبيق باتشات الإيميل تلقائياً

أمر `git am` (Apply Mailbox) يقرأ سلسلة ملفات الباتشات التي جاية من الإيميل ويحولها لكوميتات كاملة في مشروعك بنفس اسم الكاتب وتاريخه ورسالته الأصلية.

```
git am أوامر
```

```
# تطبيق كل ملفات الباتش بالترتيب
git am *.patch

# conflict: لو حصل
git am --resolved      # بعد ما تجل التعارض يدوياً
git am --skip          # لتخطي الباتش دي
git am --abort          # لإلغاء كل حاجة والرجوع لنقطة الصفر
```

Diffing

32. Git Range-Diff — مقارنة نسختين من سلسلة Commits

أمر `git range-diff` يقارن نسختين من سلسلة كوميتات (مثلاً قبل وبعد ما عملت Rebase أو بعد مراجعة ال PR) عشان يوريك إيه اللي اتغير في كل كوميت تحديداً.

```
git range-diff استخدام

# Rebase مقارنة سلسلة الكوميتات قبل وبعد ال
git range-diff main..feature/old-auth main..feature/rebased-auth
```

Plumbing

33. Git Merge-Base — تحديد السلف والأصل المشترك

أمر `git merge-base` بيطلعك أول كوميت مشترك اتفرع منه الفرعين. ده الأساس الخوارزمي اللي Git بيبنى عليه قرارات ال Way-3 Merge وال Rebase في الكواليس.

```
استخراج الأصل المشترك

# feature و main معرفة الكوميت المشترك بين
git merge-base main feature/payment
```

Plumbing

34. Git Rev-Parse — فك وترجمة المراجع للسكربتات

أمر `git rev-parse` أداة منخفضة المستوى (Plumbing) بنستخدمها في السكربتات والأتمتة عشان نترجم أسماء الفروع أو كلمة `HEAD` لل SHA-1 Hash الكامل بتاعها، أو نعرف مسار فولدر المشروع.

```
git rev-parse أمثلة

# الحالي HEAD الكامل لل Hash استخراج ال
git rev-parse HEAD

# Git التأكد برمجياً إننا واقفين جوه مشروع
git rev-parse --is-inside-work-tree

# معرفة المسار الكامل لفولدر المشروع الرئيسي
git rev-parse --show-toplevel
```

أمر `git reflog` هو الحارس الأمين وطوق النجاة بتاعك على لابتوبك؛ يسجل أي حركة لمؤشر **HEAD** أو الفروع (سواء عملت `commit`, `checkout`, `switch`, `rebase`, `reset`). مفيش حاجة بتضيع في Git طالما معاك ال Reflog!

```
HEAD@{0}: reset: moving to HEAD~3 (!مصابة: مسحت 3 كوميتات بالخطأ)
HEAD@{1}: commit: feat(order): implement order checkout endpoint (d8a4f12)
HEAD@{2}: commit: feat(order): add order repository (7c2b3e4)
HEAD@{3}: switch: moving from feature/auth to main
```

الشكل 35.1: سجل حركات مؤشر HEAD جوه ال Reflog

سيناريو الإنقاذ السحري (استرجاع Commits اتمسحت بالخطأ):

عملت بالخطأ `git reset --hard HEAD~3` وكل شغلك اختفى، أو مسحت فرع بالغلط بـ `git branch -D`:

Reflog استرجاع الشغل الضائع بـ

```
# افتح سجل الحركات وشوف كنت واقف فين قبل الكارثة. 1.
git reflog

# بالمللي Reset ارجع بالزمن لنقطة ما قبل الـ 2.
git reset --hard HEAD@{1}

# أو أنشئ فرع جديد ينطلق من الكوميت اللي كان ضائع:
git branch recovered-feature HEAD@{1}
```

أمر `git fsck` (File System Consistency Check) بيكشف على صحة قاعدة بيانات Git ويتأكد إن مفيش ملفات تالفة أو كائنات معلقة ملهاش صاحب (Dangling/Unreachable Objects).

فحص صحة المستودع

```
# فحص كامل للمستودع وكشف الكائنات المعلقة
git fsck --full --unreachable
```

أمر `git gc` (Garbage Collection) بيضغط الملفات الفردية المبعثرة (Loose Objects) ويجمعها في ملفات مضغوطة ومفهرسة اسمها (Packfiles) عشان يسرع أداء Git ويوفر مساحة الهارد.

تحسين وضغط المستودع

```
# تشغيل التنظيف والضغط العادي
git gc

# تشغيل ضغط مكثف جداً للمشاريع الكبيرة
git gc --aggressive --prune=now
```


أمر `git prune` منخفض المستوى، يمسح الكائنات القديمة المعلقة اللي مفيش أي فرع بيشارور عليها نهائياً (عادة `git gc` بيشفله تلقائياً).

تنظيف الكائنات الميتة

```
# تجربة جافة تشوف إيه اللي هيتمسح
git prune --dry-run --verbose

# مسح الكائنات المعلقة اللي فات عليها أكثر من أسبوعين
git prune --expire=2.weeks.ago
```

في الشركات الكبيرة، المطور بيوقع على الكوميترات بتاعته بمفتاح مشفر (GPG أو SSH) عشان يثبت هويته ومحدث ينتحل شخصيته.

إدارة التوقيعات المشفرة

```
# موقع رقمياً Commit إنشاء
git commit -S -m "feat(security): enable TLS 1.3"

# التحقق من صحة التوقيع
git verify-commit HEAD

# عرض تاريخ الكوميترات مع حالة التوقيع
git log --show-signature
```

هي الأداة الرسمية الموصى بها لإعادة كتابة التاريخ العميق ومسح ملفات حساسة أو كلمات سر اترفعت بالغلط زمان في أول المشروع.

🚨 قاعدة أمنية صارمة بخصوص الأسرار المسربة

مسح الـ **Secret** من تاريخ **Git** مش معناه إنه بقى آمن! بمجرد ما الـ **Key** ارفع على السيرفر، اعتبره اتسرق فوراً وادخل لوحة التحكم واعمله **Revoke** و **Rotate** ونزل مفتاح جديد، وبعدة طهر الـ **Repo**.

git-filter-repo تطهير المستودع بـ

```
# تثبيت الأداة:
pip install git-filter-repo

# مسح ملف سري من كامل تاريخ المشروع من أول يوم لآخره
git filter-repo --path src/appsettings.Production.json --invert-paths

# مسح نص كلمة سر معينة من جوه كل الملفات واستبدالها بنص آمن
git filter-repo --replace-text expressions.txt

# رفع التاريخ الجديد للسيفر بالإجبار (محتاج أدمن)
git push origin --force --all
git push origin --force --tags
```

Architecture

41. Git Submodules — ربط مشاريع فرعية جوه مشروعك

ميزة **Git Submodules** بتخليك تحط مستودع Git كامل جوه فولدر فرعي في مشروعك وتثبتته عند كوميت محدد. الشركات بتستخدمها عشان تشارك مكتبات مشتركة (Shared Libraries) بين مشاريع مختلفة.

Submodules أوامر الـ

```
# إضافة مشروع فرعي لمشروعك 1.
git submodule add https://github.com/company/CommonLogging.git lib/CommonLogging

# بتاعته في خطوة واحدة submodules استنساخ مشروع بجميع الـ 2.
git clone --recurse-submodules https://github.com/company/MainApp.git

# في مشروع نزلته من غيرهم submodules تحديث الـ 3.
git submodule update --init --recursive
```

Architecture

42. Git Subtree — دمج كود المشاريع المشتركة مباشرة

Git Subtree بديل أسهل لـ Submodules، بدمج كود المشروع الخارجي مباشرة جوه الفولدر والتاريخ كأنه جزء أصيل من مشروعك، بدون وجع دماغ ملفات `.gitmodules`.

Git Subtree استخدام

```
# إضافة كود مشروع خارجي جوه فولدر محدد
git subtree add --prefix=src/SharedAuth https://github.com/company/SharedAuth.git main --squash
```

Large Files

43. Git LFS — تخزين الملفات والميديا الضخمة

Git LFS (Large File Storage) امتداد رسمي بيحل مشكلة الملفات الكبيرة (فيديوهات، ملفات 3D، ملفات ذكاء اصطناعي ونماذج AI) عن طريق إنه يحط ملف نصي صغير مكانه كـ Pointer، ويرفع الملف الثقيل على سيرفر تخزين مخصص.

تهيئة Git LFS

```
# تفعيل LFS
git lfs install

# تتبع أنواع الملفات الكبيرة
git lfs track "*.psd"
git lfs track "models/*.onnx"

git add .gitattributes
git commit -m "chore: track models with Git LFS"
```

Offline

44. Git Bundle — نقل المشاريع بفلاشة من غير نت

أمر `git bundle` يضغط مشروعك بكل فروعه وكائناته وتاريخه في ملف واحد `.bundle`، تقدر تأخذه على فلاشة USB وتنقل المشروع لجهاز معزول تماماً عن النت (Air-gapped).

حزم ونقل المشاريع أوفلاين

```
# في ملف واحد main حزم فرع
git bundle create repo-main.bundle main

# على جهاز أوفلاين Bundle استنساخ المشروع من ملف الـ
git clone repo-main.bundle MyOfflineProject
```

Concepts

45. Git Archive vs Source Export — المستودع الحي ضد الأرشفة

Source Export / Archive (الأرشفة النقي)

- شجرة ملفات الكود بس في لحظة معينة.
- مفهوش فولدر `.git` ولا أي تاريخ سابق.
- حجمه صغير ومناسب لتسليم النسخ للعميل أو بناء الـ Docker.

Git Repository (المستودع الحي)

- فيه فولدر `.git` وقاعدة البيانات والتاريخ كله.
- تقدر تعمل Commits وفروع وتراجع وتعمل Push/Pull.
- حجمه أكبر عشان شايك تاريخ التغييرات عبر الزمن.

Troubleshooting

46. Git Check-Ignore — كشف سبب تجاهل ملف معين

أمر `git check-ignore` بيعرفك بدقة ليه Git متجاهل ملف معين، ويطلعلك اسم ملف `.gitignore` ورقم السطر اللي مسبب التجاهل ده.

.gitignore استكشاف أخطاء

```
# معرفة سبب تجاهل الملف
git check-ignore -v src/appsettings.Development.json

# النتيجة:
# .gitignore:18:appsettings.*.json      src/appsettings.Development.json
```

Plumbing

47. Git LS-Files — فحص محتويات الفهرس الداخلي (Index)

أمر `git ls-files` بيعرض الملفات المسجلة حالياً جوه الـ Index والـ Staging وحالتها الفعلية.

استعراض الفهرس

```
# عرض كل الملفات المتتبعه في الفهرس
git ls-files

# عرض الملفات المعدلة اللي لسه متجهزتش
git ls-files -m

# .gitignore. عرض الملفات المتجاهلة بـ
git ls-files -i --exclude-standard
```

Plumbing

48. Git LS-Tree — فحص شجرة الملفات والهاشات

أمر `git ls-tree` يعرض هيكل المجلدات والملفات وال SHA Hashes لكل ملف جوه Commit محدد، زي أمر `ls` في لينكس.

فحص شجرة الكائنات

```
# عرض شجرة الملفات لآخر كوميت
git ls-tree HEAD

# (Recursive) عرض الشجرة بكل الفولدرات الفرعية
git ls-tree -r -t HEAD
```

Plumbing

49. Git Show-Ref — استعراض كل المؤشرات والمراجع

أمر `git show-ref` يبطع كل الفروع والوسوم المحلية والبعيدة مع ال Hashes اللي بتشاور عليها.

استعراض المراجع

```
# عرض جميع المراجع
git show-ref

# عرض وسوم الإصدارات فقط
git show-ref --tags
```

Scripting

50. Git For-Each-Ref — استخراج تقارير الفروع بالأتمتة

أمر `git for-each-ref` هو الأداة المفضلة في سكربتات الأتمتة عشان تفرز وتنسق بيانات الفروع وال Tags حسب ما تحب.

استخراج تقارير الفروع

```
# ترتيب الفروع المحلية حسب تاريخ آخر تعديل تنازلياً مع اسم المطور
git for-each-ref --sort=-committerdate refs/heads/ --format='%(committerdate:short) | %(authorname) | %(refname:short)'
```

Plumbing

51. Git Cat-File — تشريح كائنات (Blob, Tree, Commit)

في أعماق Git، هو عبارة عن **Key-Value Store** مبني على 4 كائنات رئيسية:

- **Blob**: بيخزن محتوى الملف البرمجي بس.

- **Tree**: يمثّل الفولدر ويحتوي على قائمة بال Blobs وال Trees الفرعية وأسماء الملفات.
- **Commit**: يبيّن على Tree، وفيه بيانات الكاتب والتاريخ والرسالة ومؤشر ال Parent.
- **Tag**: علامة مميزة على كوميت معين مع بيانات التوثيق والتوقيع.

Git تشريح كائنات

```
# 1. معرفة نوع الكائن (blob, tree, commit, tag)
git cat-file -t d8a4f12

# 2. طباعة المحتوى الحقيقي المقروء للكائن (Pretty-print)
git cat-file -p HEAD
```

Plumbing

52. Git Hash-Object — حساب وتوليد الهاش التشفيري

أمر `git hash-object` يحسب المعرف التشفيري (SHA-1 / SHA-256) لأي ملف أو نص، ومعه خيار تخزينه ك Blob حقيقي جوه `.git/objects`.

Blobs حساب وتخزين الـ

```
# Git حساب الهاش وحفظ الملف في قاعدة بيانات
git hash-object -w src/Models/User.cs
```

Plumbing

53. Git Update-Ref — تحريك المؤشرات برمجياً

أمر `git update-ref` يعدّل القيمة اللي الفرع يبيّن عليها مباشرة لل Hash اللي انت عايزه بأمان من غير ما تعدّل الملفات يدوياً.

تحديث المؤشرات

```
# برمجياً لـ main تحريك مؤشر فرع
git update-ref refs/heads/main 7c2b3e4
```

Plumbing

54. Git Read-Tree — قراءة الشجرة جوه الفهرس

أمر `git read-tree` يقرأ شجرة معينة ويفضي محتواها ويحدث ال Index بيها على المستوى المنخفض.

تفريغ الشجرة في الفهرس

```
git read-tree 7c2b3e4
```

Plumbing

55. Git Write-Tree — تحويل الفهرس لشجرة كائنات

أمر `git write-tree` يباخذ الحالة اللي في ال Staging Area ويحولها لكائن Tree رسمي جوه قاعدة بيانات Git ويطبع الهاش بتاعه.

Tree إنشاء كائن

```
git write-tree
```

أمر `git commit-tree` يبنئ كوميت جديد عن طريق تمرير Tree Hash مع ال Parent Commit والرسالة، وده اللي `git commit` بيعمله في الكواليس!

برمجياً إنشاء Commit

```
git commit-tree e69de29 -p HEAD -m "feat: low level commit"
```

أمر `git count-objects` بيحسب عدد الكائنات المتفرقة والمضغوطة وإجمالي المساحة المستهلكة على الهارد.

Git إحصاء حجم قاعدة بيانات

```
git count-objects -vH
```

أمر `git verify-pack` بيحص ملفات الحزم المضغوطة (`.pack`) ويكتشف أكبر الملفات اللي واكله مساحة في تاريخ المشروع.

اكتشاف الملفات المتضخمة

```
git verify-pack -v .git/objects/pack/pack-*.idx | sort -k 3 -n -r | head -n 10
```

أمر `git check-ref-format` بيتأكد إن الاسم المقترح لل Branch مبيخالفش قواعد التسمية في Git ومفهوش رموز ممنوعة (زي المسافات أو `..` أو `~`).

التحقق من صحة الاسم

```
git check-ref-format --branch "feature/login-v1" && echo "Valid Name"
```

Git Hooks هي سكريبتات بتشتغل أوتوماتيك أول ما يحصل حدث معين في Git (زي قبل الكوميت أو قبل ال Push) جوه فولدر `.git/hooks/`.

أشهر أنواع ال Hooks في الشغل:

- `pre-commit` : بيتشتغل قبل تسجيل الكوميت عشان يشيك على فورمات الكود (Linter) ويتأكد مفيش Secrets.
- `commit-msg` : بيتأكد إن رسالة الكوميت مكتوبة صح حسب معايير التيم.
- `pre-push` : بيتشتغل قبل الرفع للسيرفر عشان يشغل ال Unit Tests والتأكد إن ال Build سليم 100%.

سكربت عملي لمشروع (Pre-Push Hook) ASP.NET Core:

ملف `.git/hooks/pre-push` يعمل تيسر للكود ويمنع ال Push لو في أي اختبار ساقط:

NET. لمشروع `.git/hooks/pre-push` سكربت

```
#!/bin/sh
echo "🔍 Running pre-push checks for ASP.NET Core..."

# 1. التحقق من تنسيق الكود
dotnet format --verify-no-changes
if [ $? -ne 0 ]; then
    echo "❌ Pre-push failed: Code formatting issues found. Run 'dotnet format' first."
    exit 1
fi

# 2. تشغيل الـ Unit Tests
dotnet test --configuration Release --no-build
if [ $? -ne 0 ]; then
    echo "❌ Pre-push failed: Unit tests are failing! Fix tests before pushing."
    exit 1
fi

echo "✅ All tests passed! Pushing to remote..."
exit 0
```

Configuration & Security

61. Git Ignore & Secrets — حماية الأسرار وملفات NET.

ملف `.gitignore` يحدد الحاجات اللي Git ملوش دعوة بيها وميرفعهاش على السيرفر إطلاقاً.

ملف `.gitignore` نموذجي لمشاريع ASP.NET Core:

ASP.NET Core لمشروع `.gitignore` ملف

```
# ملفات البناء والتجميع المؤقتة
[Bb]in/
[Oo]bj/
[Tt]est[Rr]esults/

# إعدادات المستخدم الشخصية
*.user
.vs/
.vscode/*
!.vscode/settings.json

# ملفات البيئة والأسرار الحساسة (مهم جداً!)
.env
.env.*
appsettings.Production.json
*.pfx
*.key
*.pem
```

🚨 قاعدة ذهبية: ممنوع رفع الأسرار البرمجية إطلاقاً

إياك ترفع كلمات السر أو سلاسل الاتصال (Connection Strings) أو مفاتيح الـ API في الـ Repo! استخدم دائماً **User Secrets** (**dotnet user-secrets**) وأنت شغال لوكال، واستخدم **Azure Key Vault** أو **Environment Variables** في السيرفر.

Configuration

62. Git Attributes — تطبيق الـ Line Endings (CRLF vs LF)

ملف `.gitattributes` ييوجد سلوك الملفات بين الويندوز واللينكس والماك عشان محدش يجيله تعارضات بسبب الـ Line Endings.

القياسي `.gitattributes` ملف

```
# التطبيق التلقائي لنهايات الأسطر
* text=auto

# (CI/CD المتوافق مع لينكس ودوكر والـ) LF إجبار ملفات الكود تبقى بنظام
*.cs text eol=lf
*.json text eol=lf
*.csproj text eol=lf
*.md text eol=lf

# الملفات الثنائية (الصور والمكتبات)
*.png binary
*.dll binary
*.exe binary
```

Productivity

63. Git Aliases — اختصارات لزيادة السرعة والإنتاجية

الـ Aliases بتخليك تعمل اختصارات سريعة للأوامر الطويلة عشان تنجز وتكتب بسرعة.

(Aliases) إعداد اختصارات الإنتاجية

```
# اختصارات يومية سريعة
git config --global alias.st status
git config --global alias.sw switch
git config --global alias.br branch
git config --global alias.ci commit

# رسم الشجرة بالألوان واختصار الكوميتات
git config --global alias.lg "log --color --graph --pretty=format:'%Cred%h%Creset -%C(yellow)%d%Creset %s %Cgreen(%cr) %C(bold blue)<%an>%Creset' --abbrev-commit"

# التراجع عن آخر كوميت مع إبقاء الكود في مكانه
git config --global alias.uncommit "reset --soft HEAD~1"
```

Team Collaboration

64. Git Workflows — مقارنة مسارات ونماذج شغل التيم

الفرق البرمجية بتنظم شغلها وفروعها بأشكال ونماذج مختلفة:

النموذج (Workflow)	هيكل الفروع	المزايا	يستخدم في؟
Feature Branch Workflow	<code>main</code> + فروع <code>feature/*</code> بنتدمج بـ PR.	بسيط جداً ومرن ويضمن مراجعة الكود قبل الدمج.	أغلب الشركات والفرق المتوسطة ومشاريع الويب.
Git Flow	<code>main</code> , <code>develop</code> , <code>feature/*</code> , <code>release/*</code> , <code>hotfix/*</code>	تنظيم صارم جداً لمواعيد الـ Releases ودورات التيسر.	البنوك والأنظمة الكبيرة المعقدة (Enterprise).
Trunk-Based Development	فرع <code>main</code> واحد مع فروع قصيرة جداً (ساعات) و Feature Flags.	سرعة قياسية في النشر، تكامل مستمر حقيقي، مفيش Merge Conflicts.	شركات التقنية العالمية الكبيرة وبيئات الـ Microservices.

Hands-on Scenario

65. Real Backend .NET Scenario — سيناريو عملي لـ 4 مطورين

تخيل إنك شغال في تيم من 4 ديفيلوبرز بتبنوا E-Commerce Web API باستخدام ASP.NET Core 9:

الفروع في المشروع:

- `main` : فرع الإنتاج المستقر (Production).
- `develop` : فرع التجميع اليومي المشترك.
- `feature/authentication` : فرع الميزة بتاعتك أنت شخصياً.

رحلة الميزة الكاملة من أول ما بدأت لحد ما نزلت في البرودكشن:

دورة تطوير الميزة الكاملة

```
# 1. develop تنزيل المشروع والانتقال لفرع
git clone https://github.com/company/EcommerceBackend.git
cd EcommerceBackend
git switch develop

# 2. إنشاء فرع الميزة بتاعتك والانتقال له
git switch -c feature/authentication

# 3. وعملته فحص وتجهيز JWT كتبت كود الـ
git status
git add src/Controllers/AuthController.cs src/Services/JwtService.cs

# 4. تسجيل الكوميت برسالة احترافية
git commit -m "feat(auth): implement JWT token generation and validation"

# 5. وأنت شغال develop مزامنة التعديلات الي زمايلك رفعوها على:
git fetch origin
git rebase origin/develop

# 6. Conflict: لو ظهر) حل التعارض:
# وصلح الكود المتعارض ثم Program.cs افتح:
git add src/Program.cs
git rebase --continue

# 7. رفع فرعك للـ GitHub
git push -u origin feature/authentication

# 8. develop لفرع Squash and Merge ومراجعته من التيم، ثم عمل Pull Request فتح.

# 9. مسح الفرع المحلي بعد ما خلص
git switch develop
git pull --rebase
git branch -d feature/authentication
```

طوارئ البرودكشن: خطف باج بـ Cherry-Pick وتوثيق Release Tag:

إصلاح الطوارئ وتوثيق التاج

```
# f8a12d3 بهاش develop إصلاح باج عاجل في العملات تم تصليحه في
git switch main
git pull
git cherry-pick f8a12d3
dotnet test
git push origin main
git tag -a v1.0.1 -m "Hotfix: resolve currency conversion bug in production"
git push origin v1.0.1
```

المستوى (Level)	طبيعة الأوامر	أهم الأوامر	بتستخدمها كام مرة؟	مستوى الخطورة
Beginner / Daily (Porcelain)	الأوامر الأساسية اليومية لكل مطور.	<code>commit</code> , <code>add</code> , <code>status</code> , <code>clone</code> , <code>init</code> <code>switch</code> , <code>branch</code> , <code>fetch</code> , <code>pull</code> , <code>push</code> <code>stash</code> , <code>diff</code> , <code>log</code> , <code>merge</code>	طول اليوم (100%)	منخفض جداً (Safe)
Intermediate	أوامر إدارة التاريخ وحل المشاكل المعقدة.	<code>reset</code> , <code>revert</code> , <code>cherry-pick</code> , <code>rebase</code> <code>bisect</code> , <code>grep</code> , <code>blame</code> , <code>worktree</code> , <code>tag</code> <code>range-diff</code> , <code>shortlog</code>	أسبوعياً وعند الحاجة	متوسط إلى مرتفع
Advanced / Plumbing	أوامر النواة منخفضة المستوى والسكريبتات.	<code>read-tree</code> , <code>hash-object</code> , <code>cat-file</code> <code>update-ref</code> , <code>commit-tree</code> , <code>write-tree</code> <code>ls-files</code> , <code>ls-tree</code> , <code>rev-parse</code> <code>verify-pack</code>	للأتمتة وبناء الأدوات	مرتفع (محتاجة فهم عميق)

الخطورة	التصنيف	مثال تطبيقي (Example)	وظيفته (Purpose)	الأمر (Command)
Safe	Daily	git init	تهيئة مستودع محلي جديد	git init
Safe	Daily	git clone <url>	تنزيل مشروع كامل من السيرفر	git clone
Safe	Daily	git status -s	فحص حالة الملفات والتجهيز	git status
Safe	Daily	git add -A	تجهيز التعديلات في الـ Staging	git add
Safe	Daily	git commit -m "feat: login"	تسجيل لقطة دائمة جديدة	git commit
Safe	Daily	git diff --staged	عرض الفروقات بين الملفات سطر بسطر	git diff
Safe	Daily	git restore app.cs	إلغاء تعديلات ملفات غير مسجلة	git restore
Safe	Daily	git branch -a	إنشاء وعرض وحذف الفروع	git branch
Safe	Daily	git switch -c feat/order	التنقل بين الفروع وإنشائها بوضوح	git switch
Low	Daily	git merge feat/order	دمج فرعين مع بعض	git merge
Medium	Intermediate	git rebase -i HEAD~3	إعادة تأسيس الفروع لتاريخ خطي نظيف	git rebase
Safe	Daily	git stash pop	ركن التعديلات غير المكتملة على جنب	git stash
Safe	Daily	git fetch --all --prune	جلب التحديثات من غير دمجها في كودك	git fetch
Low	Daily	git pull --rebase	جلب ودمج التحديثات (Fetch + Rebase/Merge)	git pull
Medium	Daily	git push --force-with-lease	رفع الشغل للسيرفر بأمان	git push
Low	Intermediate	git cherry-pick e4a19b	خطف commit محدد من فرع لفرع ثاني	git cherry-pick
High	Intermediate	git reset --soft HEAD~1	الرجوع في الكوميتات وتعديل التاريخ	git reset
Safe	Intermediate	git revert d8a4f1	إلغاء كوميت بعمل كوميت مضاد آمن	git revert
Safe	Intermediate	git bisect start	صيد الباجات بالبحث الثنائي	git bisect
Safe	Advanced	git worktree add ../hotfix	فتح مساحات عمل لفروع ثانية بالتوازي	git worktree
Safe	Intermediate	git reflog	سجل الأمان الشامل لكل حركات المؤشرات	git reflog
Safe	Plumbing	git cat-file -p HEAD	تشرح كائنات Git الداخلية	git cat-file
Safe	Plumbing	git hash-object -w file.cs	حساب وتخزين الهاش المشفر للكائنات	git hash-object
High	Intermediate	git clean -fd	مسح الملفات غير المتتبعة نهائياً من الهارد	git clean

أوامر الشغل والتفريع اليومية:

1. `git status` : بوصلتك اليومية لمعرفة حالة الكود.
2. `git add` : تجهيز التعديلات باحتراف.
3. `git commit` : تسجيل اللقطات البرمجية.
4. `git log` : استكشاف وقراءة تاريخ المشروع.
5. `git diff` : مراجعة دقيقة لكل سطر اتعدل.
6. `git branch` : تنظيم مسارات التطوير.
7. `git switch` : التنقل النظيف بين الفروع.
8. `git merge` : دمج الشغل المستقر في المسار العام.
9. `git rebase` : الحفاظ على تاريخ خطي مستقيم ونظيف.
10. `git stash` : ركن الشغل بسرعة وقت الطوارئ.

أوامر المزامنة والإنقاذ والتراجع:

11. `git fetch` : استكشاف اللي حاصل على السيرفر بأمان.
12. `git pull` : مزامنة كود زمايلك عندك.
13. `git push` : رفع ونشر شغلك على السيرفر.
14. `git restore` : التراجع السريع عن تعديلات الملفات.
15. `git reset` : تصحيح وتنظيم الكوميتات المحلية.
16. `git revert` : التراجع الآمن عن كود نزل على فروع عامة.
17. `git cherry-pick` : نقل الإصلاحات السريعة بين الفروع.
18. `git reflog` : طوق النجاة واسترجاع الشغل الضائع.
19. `git blame` : فهم أسباب وتاريخ كل سطر برمجي.
20. `git bisect` : صائد الباجات والتراجعات الخفية.
21. `git worktree` : الشغل على كذا فرع بالتوازي.

Troubleshooting

69. Common Git Mistakes — أشهر المصايب وإزاي تحلها

2. عمل Rebase على Shared Branch (زي main)

المصيبة: تغيير الهاشات ولخبطة كل مستودعات التيم.
القاعدة: اعمل Rebase لفروعك المحلية بس، واستخدم Merge للفروع العامة.

1. استخدام `git push --force` على فرع مشترك

المصيبة: مسح شغل زمايلك على السيرفر.
الحل: استخدم دائماً `git push --force-with-lease` اللي بتمنع المسح لو في كود جديد.

4. تشغيل `git clean -f` بدون تجربة جافة

المصيبة: مسح ملفات جديدة نهائياً من الهارد.
الحل: لازم تشغيل `git clean -nd` الأول تتأكد إيه اللي هيتمسح.

3. تشغيل `git reset --hard` وفقدان كود مهم

المصيبة: مسح ملفات لسه مكنتش اتحفظت.
الحل: افتح `git reflog` وحدد الكوميت واعمل `git reset --hard HEAD@{1}`.

Architecture Map

70. Final Git Mental Model — الخريطة المعمارية والنموذج الذهني

المخطط المعماري الشامل اللي بيلخص كل مناطق Git وتدقق الأوامر بينها:

THE COMPLETE GIT ARCHITECTURE MAP

[مساحة العمل على جهازك]

[السيرفر السحابي البعيد]



توزيع الأوامر حسب مكان عملها:

```

مساحة العمل (Working Tree): git status | git diff | git restore | git clean | git worktree
منطقة التجهيز (Index):      git add | git diff --staged | git restore --staged | git ls-files
التاريخ والتفرع المحلي:    git commit | git switch | git branch | git merge | git rebase | git cherry-pick
التراجع والأمان:          git reset (--soft/mixed/hard) | git revert | git stash | git reflog
الفحص والتشخيص:           git log | git show | git blame | git grep | git bisect | git describe
المزامنة مع السيرفر:        git remote | git fetch | git pull (--rebase) | git push (--force-with-lease)
أوامر النواة (Plumbing):    git cat-file | git hash-object | git rev-parse | git write-tree | git commit-tree
  
```

الشكل 70.1: النموذج الذهني المعماري الشامل لكافة مساحات العمل وأوامر Git

تم إعداد هذا المرجع العملي الشامل بالعامية المصرية التقنية وفق أحدث التوثيقات الرسمية لـ Git وأفضل المعايير الهندسية لشركات البرمجيات.